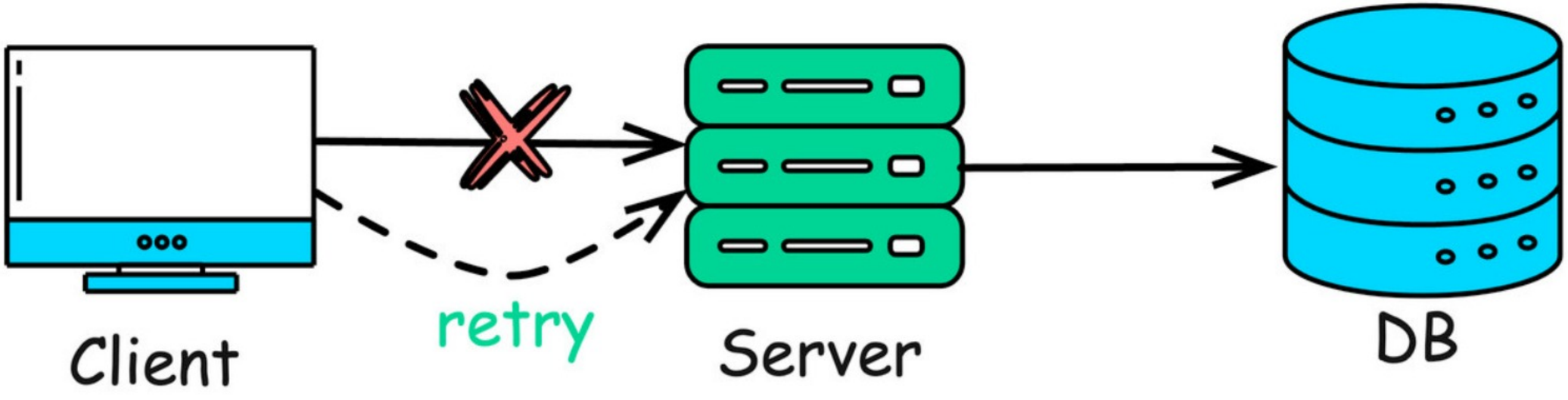
HTTP Methods Idempotency

Method	Idempotent
HEAD	YES
OPTIONS	YES
GET	YES
POST	NO
PUT	YES
PATCH	NO
TRACE	YES
DELETE	YES

Делаем запросы идемпотентными



https://www.berkansasmaz.com/every-programmer-should-know-idempotency/

Нужна гарантия, что повторный запрос не вызовет повторные эффекты

Проблемы, которые бывают

- Каждый retry = новый side-effect
- Дубли
- Человеческий фактор, дважды кнопку нажали
- Неконсистентное состояние

Пример

- Создаем заказ
- Запрос обрывается по середине
- Клиент повторяет запрос
- Создается еще один заказ

```
POST /add_row HTTP/1.1
POST /add_row HTTP/1.1    -> Adds a 2nd row
POST /add_row HTTP/1.1    -> Adds a 3rd row
```

❌

```
DELETE /idX/delete HTTP/1.1    -> Returns 200 if idX exists
DELETE /idX/delete HTTP/1.1    -> Returns 404 as it just got deleted
DELETE /idX/delete HTTP/1.1    -> Returns 404
```

✅

По-умолчанию

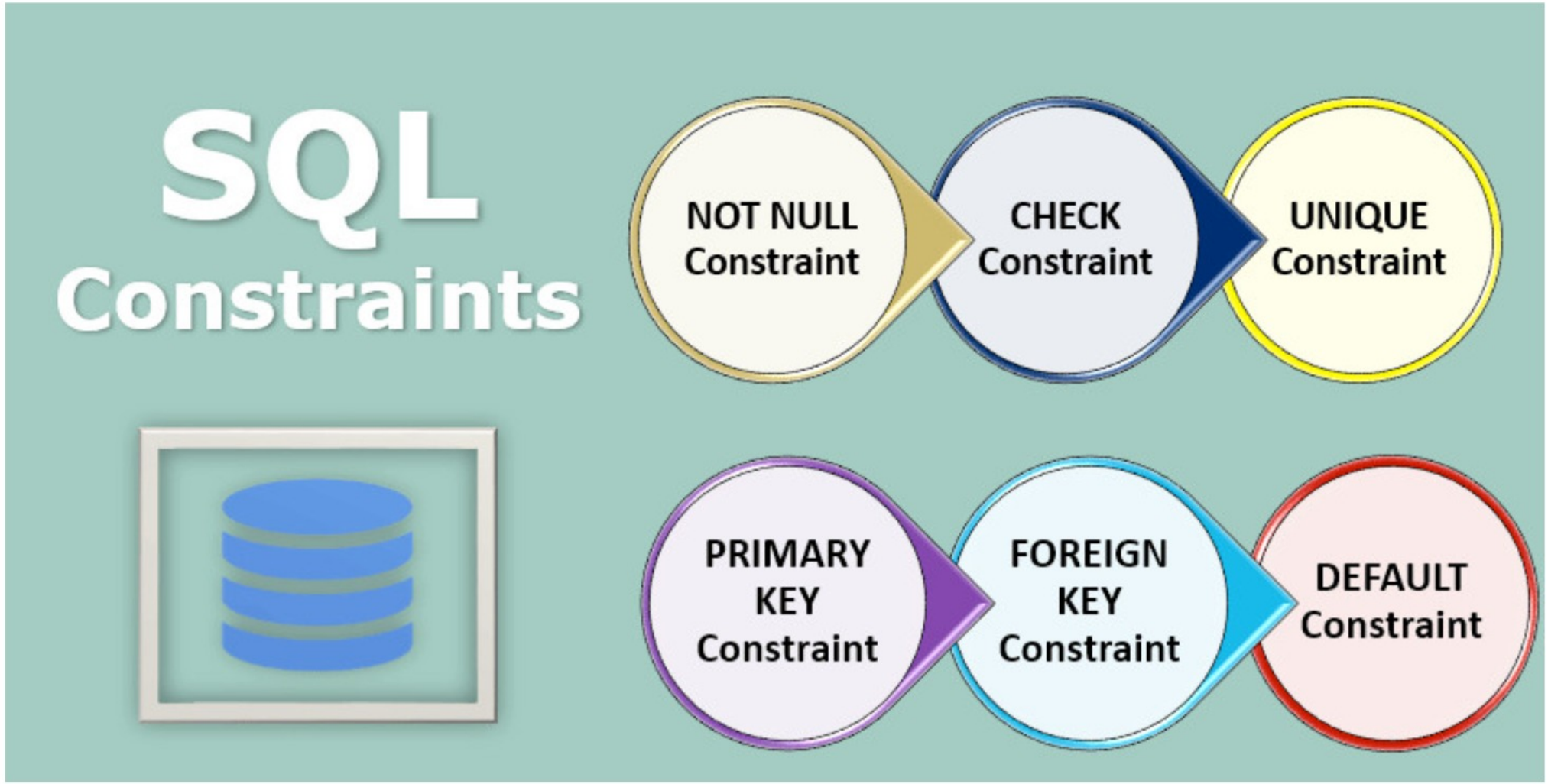
POST по умолчанию не идемпотентен — и вот тут начинаются проблемы

HTTP Methods Idempotency

Method	Idempotent
HEAD	YES
OPTIONS	YES
GET	YES
POST	NO
PUT	YES
PATCH	NO
TRACE	YES
DELETE	YES

Простой вариант: добавляем защиту

Constraint в sql, на уникальность



Example

```
1   CREATE TABLE users (
2       id BIGSERIAL PRIMARY KEY,
3       email TEXT
4       updated_at TIMESTAMP
5   );
6
7   ALTER TABLE users
8   ADD CONSTRAINT users_email_unique UNIQUE (email);
```

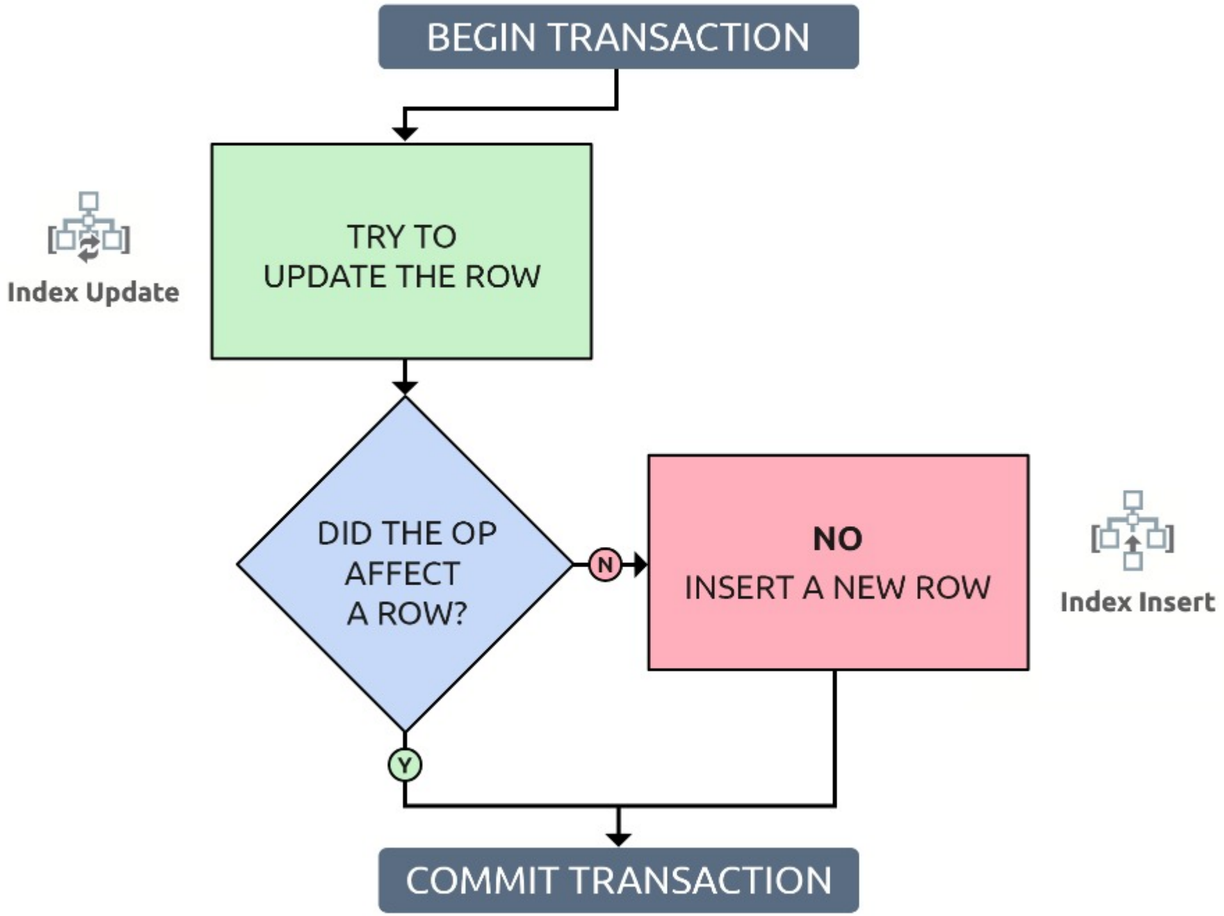
первый запрос → ✅

повтор → ❌

Для правильной обработки дубля можно ошибку пропустить

Продвинутый вариант: upsert

Вставляем запись,
если она сущетсвует, то
ничего не делаем



https://sqlperformance.com/2020/09/locking/upsert-anti-pattern

Example

```sql
INSERT INTO users (email)
VALUES ('alice@example.com')
ON DUPLICATE KEY UPDATE email = VALUES(email) and
    updated_at = NOW();
```

Костыль - обновится запись

первый запрос →

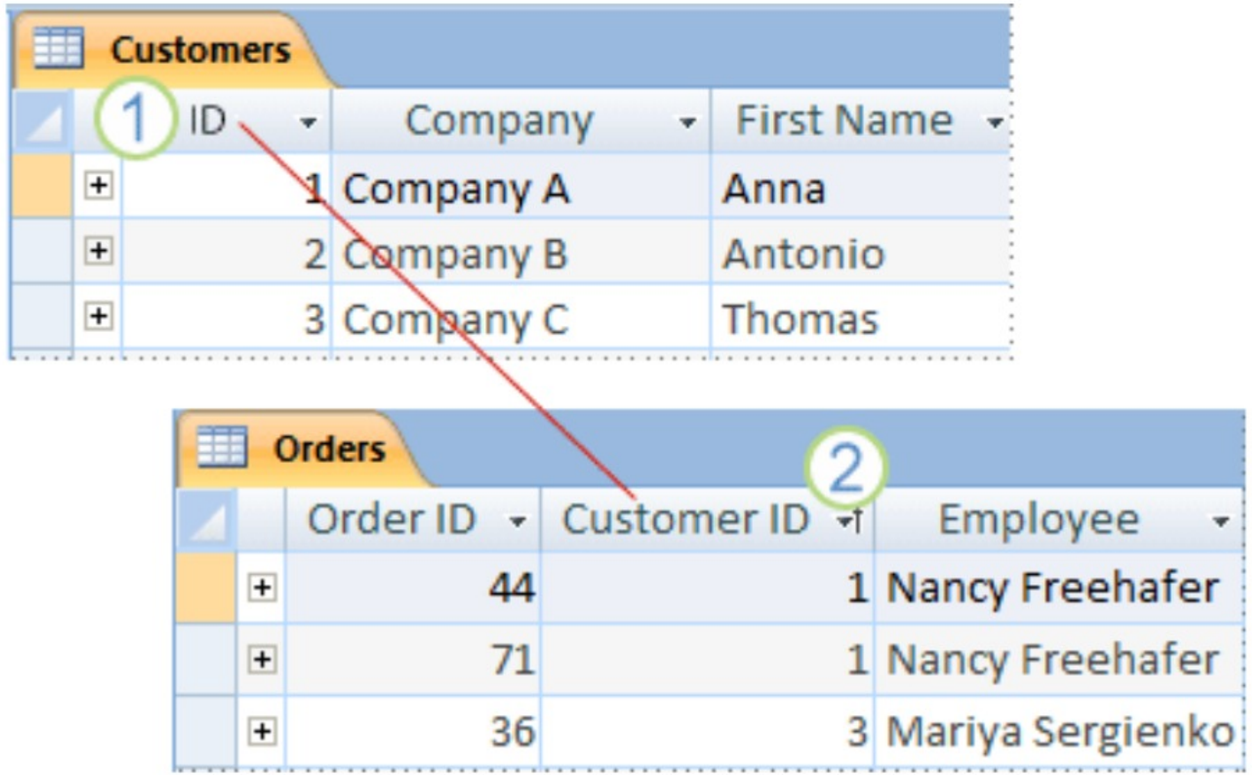
повтор →

На самом деле это не идемпотентно

Повторные запросы **снова** вызывают сайд-эффекты

Продвинутый вариант: ID на клиенте

Генерируем ID на клиенте и создаем/обновляем запись с существующим ID (идемпотентность по ID ресурса)



Ретрай не создаст проблем

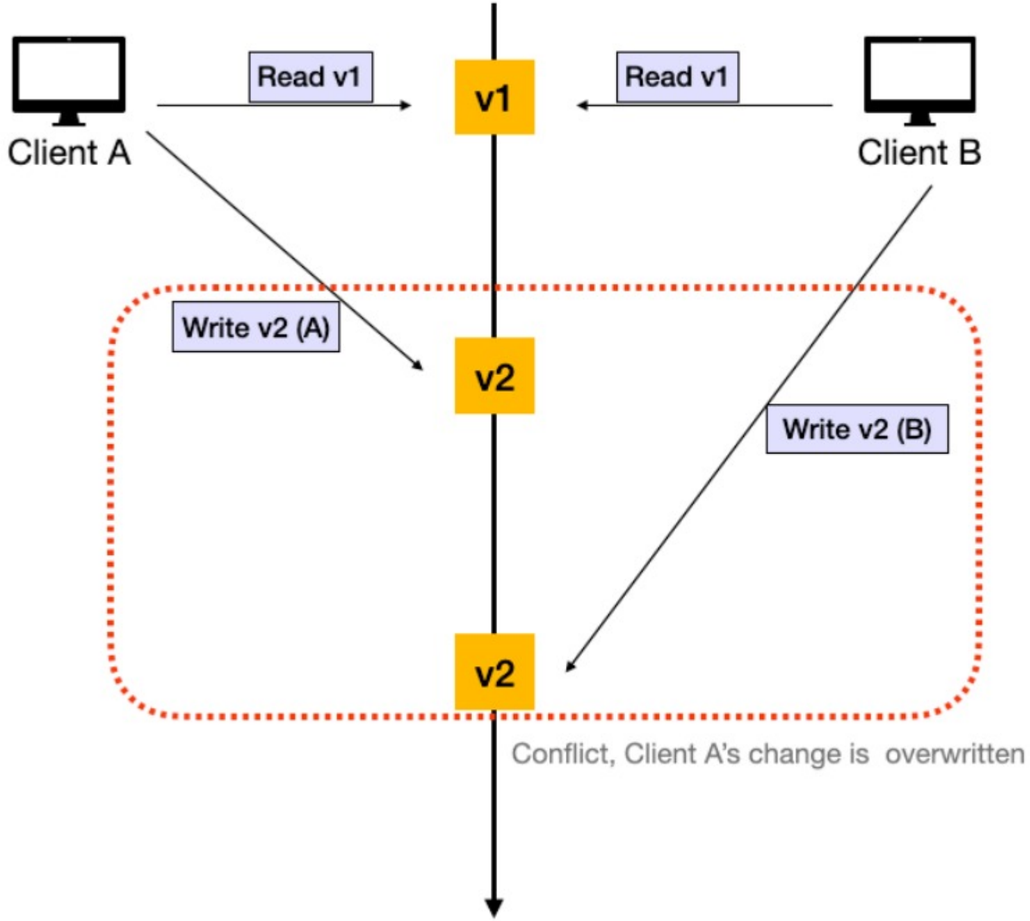
POST → плохо
PUT /orders/{id} → хорошо

https://habr.com/ru/articles/947410/

Продвинутый вариант: Optimistic lock idempotency

Обновляем данные по версии на входе, валидируем, что версия совпадает, иначе ничего не делаем

мы **НЕ применяем** операцию второй раз



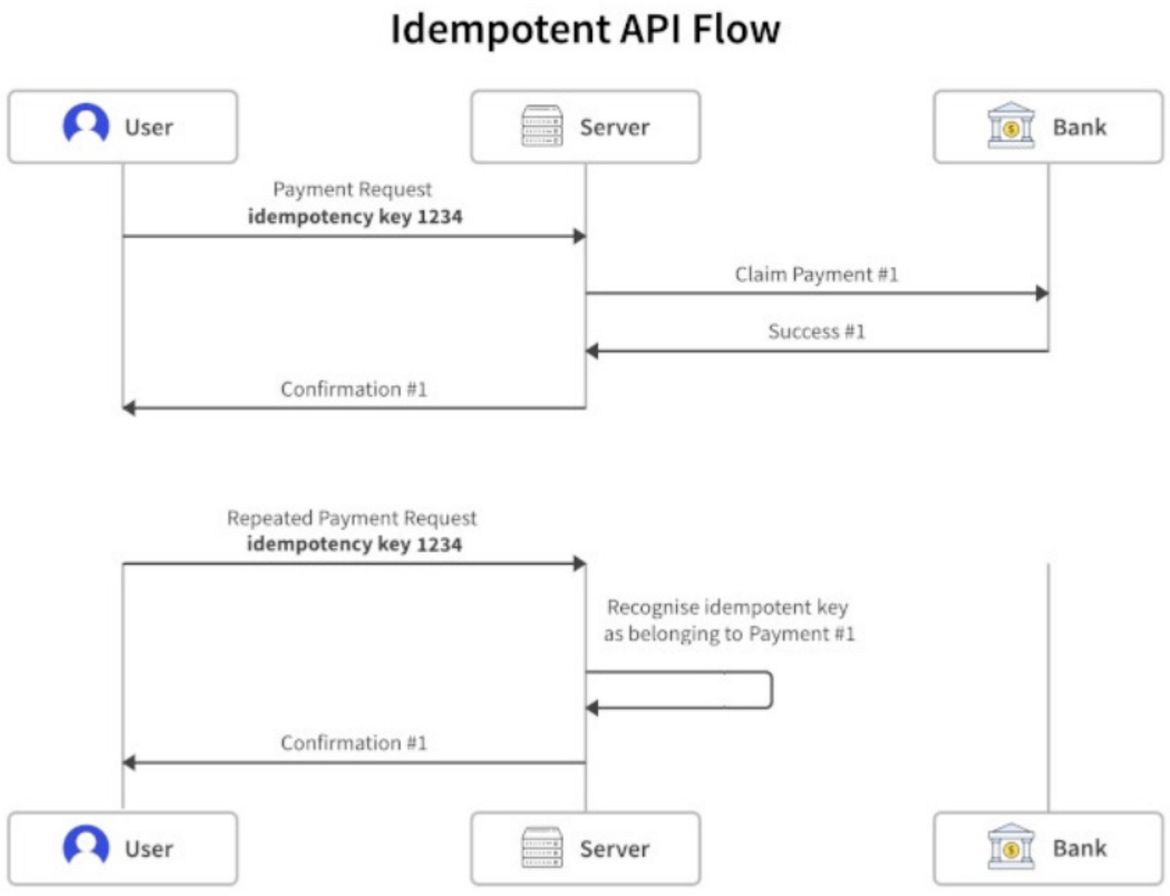
https://systemdesignschool.io/blog/optimistic-locking

Продвинутый вариант: idempotency key

Отдельный сервисный заголовок в запросе `Idempotency-Key`

Может быть отдельным полем в хранилище, а может храниться в кеше

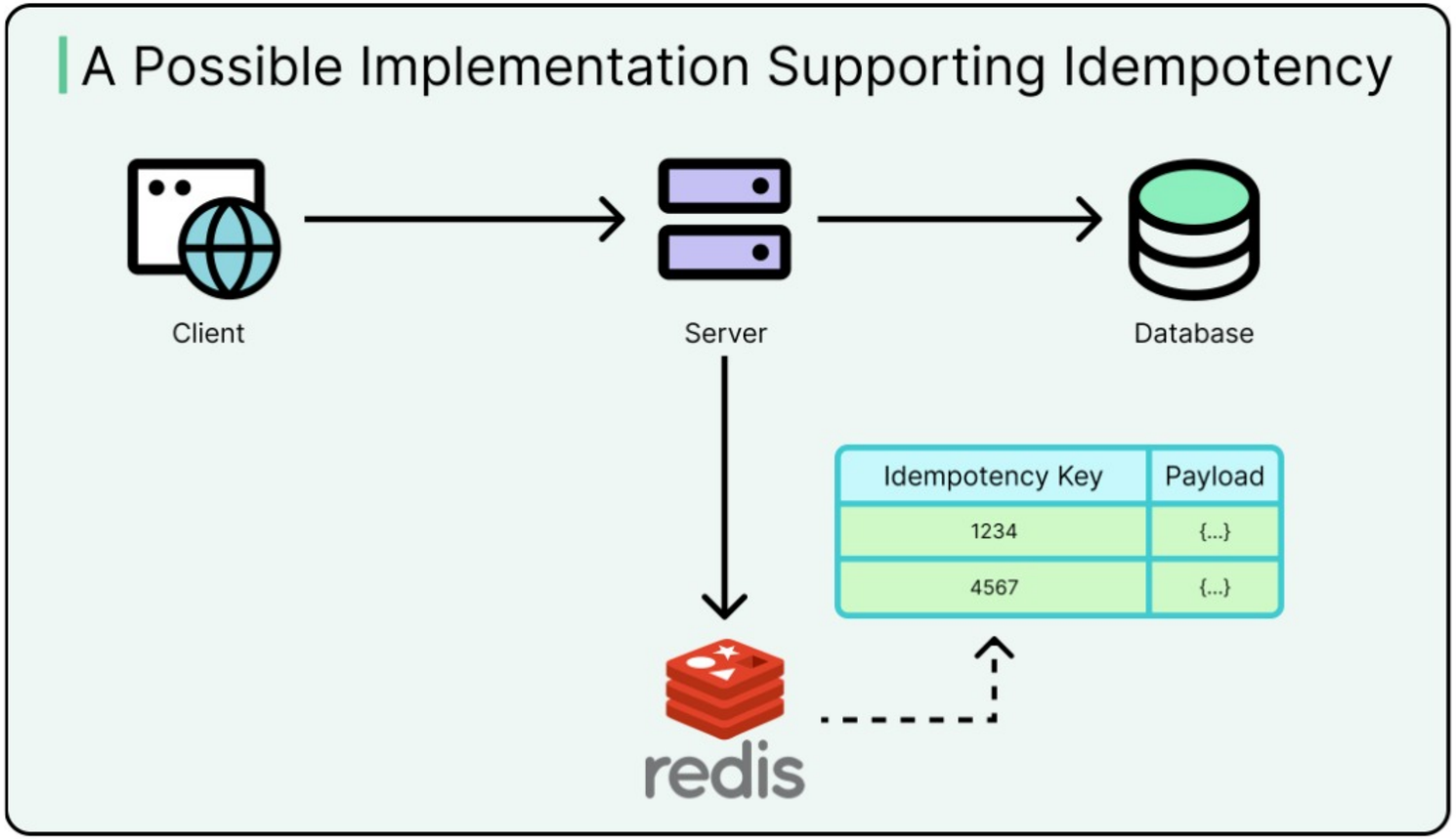
ДЕ-факто стандарт



Продвинутый вариант: event_id дедупликация

В событиях event_id и храним "окно" обработанных событий (их id), чаще всего в redis set

Если уже обрабатывали - пропускаем



Сравнение стратегий

	Стратегия	Где решает
1	Idempotency-Key	retries
2	Deterministic ID	API
3	Unique constraint	DB
4	Event deduplication	очереди
5	Upsert	user-friendly
6	Optimistic	race-condition